



**FACULTAD DE CIENCIAS EXACTAS, FÍSICAS y NATURALES**

**U.N.C.**

**Cátedra Electrónica Digital III**

**Guía de Ejercicios**

**Año 2024**

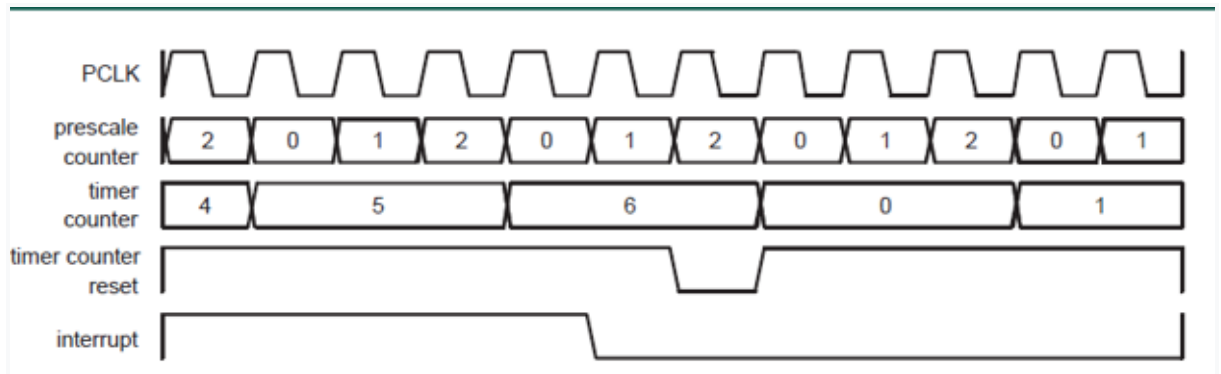
1. Una famosa empresa de calzados a incorporado a sus zapatillas 10 luces leds comandadas por un microcontrolador LPC1769 y ha pedido a su grupo de ingenieros que diseñen 2 secuencias de luces que cada cierto tiempo se vayan intercalando (secuencia A - secuencia B- secuencia A- ... ). Como todavía no se ha definido la frecuencia a la cual va a funcionar el CPU del microcontrolador, las funciones de retardos que se incorporen deben tener como parametros de entrada variables que permitan modificar el tiempo de retardo que se vaya a utilizar finalmente. Se pide escribir el código que resuelva este pedido, considerando que los leds se encuentran conectados en los puertos P0,0 al P0.9
2. En los pines P2.0 a P2.7 se encuentra conectado un display de 7 segmentos. Utilizando la variable numDisplay [10] ={0x3F, 0x06, 0x5B, 0x4F, 0x66, 0x6D, 0x7D, 0x07, 0x7F, 0x67} que codifica los números del 0 a 9 para ser mostrados en el display, realizar un programa que muestre indefinidamente la cuenta de 0 a 9 en dicho display.
3. Configurar el pin P0.4 como entrada digital con resistencia de pull down y utilizarlo para decidir si el valor representado por los pines P0.0 al P0.3 van a ser sumados o restados al valor guardado en la variable "acumulador".El valor inicial de "acumulador" es 0.
4. Considerando pulsadores normalmente abiertos conectados en un extremo a masa y en el otro directamente a las entradas P0.0 y p0.1. Realizar un programa que identifique en una variable cual o cuales pulsadores han sido presionados. Las identificaciones posibles a implementar en esta variable van a ser "ninguno", "pulsador 1", "pulsador 2", "pulsador 1 y 2".
5. Escribir un programa en C que permita realizar un promedio movil con los últimos 8 datos ingresados por el puerto 1. Considerar que cada dato es un entero signado y está formado por los 16 bits menos significativos de dicho puerto. El resultado, también de 16 bits, debe ser sacado por los pines P0.0 al P0.11 y P0.15 al P0.18. Recordar que en un promedio movil primero se descarta el dato mas viejo de los 8 datos guardados, se ingresa un nuevo dato proveniente del puerto y se realiza la nueva operación de promedio con esos 8 datos disponibles, así sucesivamente. Considerar el uso de un retardo antes de tomar una nueva muestra por el puerto.

6. Realizar un programa que configure el puerto P2.0 y P2.1 para que provoquen una interrupción por flanco de subida para el primer pin y por flanco de bajada para el segundo. Cuando la interrupción sea por P2.0 se enviará por el pin P0.0 la secuencia de bits 010011010. Si la interrupción es por P2.1 se enviará por el pin P0.1 la secuencia 011100110. Las secuencias se envían únicamente cuando se produce una interrupción, en caso contrario la salida de los pines tienen valores 1 lógicos. ¿que prioridad tienen configuradas por defecto estas interrupciones?
7. Realizar un programa que configure el puerto P0.0 y P2.0 para que provoquen una interrupción por flanco de subida. Si la interrupción es por P0.0 guardar el valor binario 100111 en la variable "auxiliar", si es por P2.0 guardar el valor binario 111001011010110.
8. Configurar la interrupción externa EINT1 para que interrumpa por flanco de bajada y la interrupción EINT2 para que interrumpa por flanco de subida. En la interrupción por flanco de bajada configurar el systick para desbordar cada 25 mseg, mientras que en la interrupción por flanco de subida configurarlo para que desborde cada 60 mseg. Considerar que EINT1 tiene mayor prioridad que EINT2.
9. Escribir un programa en C que permita sacar por los pines P1.16 al P1.23 el equivalente en ascii de "a" si la interrupción se ha realizado por interrupción de EINT1 y no hay interrupción pendiente de EINT0. Para el caso en el cual se produzca una interrupción por EINT1 y exista una interrupción pendiente de EINT0 sacar por el puerto el equivalente en ascii de "A". La interrupción de EINT1 es por el flanco de subida producido por un pulsador identificado como "tecla a/A", mientras que la interrupción EINT0 es por el nivel alto de un pulsador etiquetado como "Activación de Mayúsculas".

Nota: Valerse de la configuración de los niveles de prioridad para que la pulsación conjunta de "Activación de Mayúsculas" con "tecla a/A" de como resultado a la salida el equivalente en ascii "A".
10. Demostrar que el valor a pasar como parámetro a la función SysTick\_Config(), es la frecuencia del reloj del CPU multiplicado por el intervalo de tiempo entre interrupciones o lo que es lo mismo la frecuencia del reloj dividido la frecuencia de interrupción.
  - b. ¿El valor pasado como parámetro de entrada a la función SysTick\_Config() es el valor que finalmente se carga en el registro RELOAD?

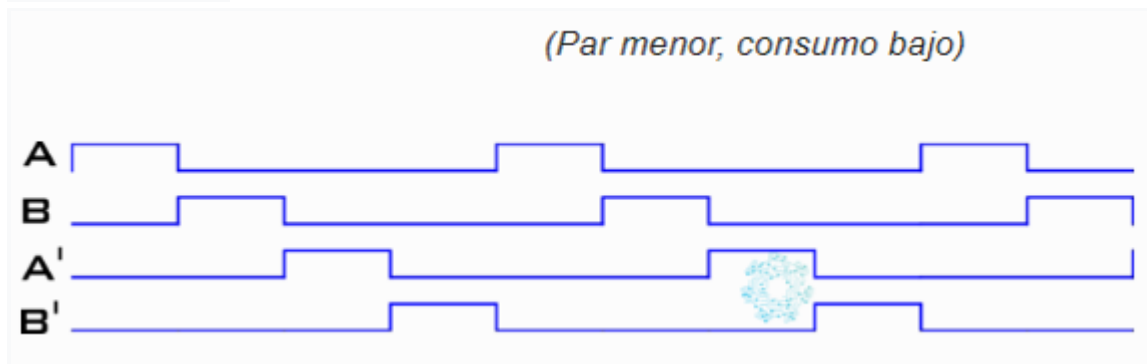
11. Configure el SysTick Timer de modo que genere una forma de onda llamada PWM tal como la que se muestra en la figura adjunta. Esta señal debe ser sacada por el pin P0.22 para que controle la intensidad de brillo del led. El periodo de la señal debe ser de 10 mseg con un duty cycle de 10%. Configure la interrupción externa EINT0 de modo que cada vez que se entre en una rutina de interrupción externa el duty cycle incremente en un 10% (1 mseg). Esto se repite hasta llegar al 100%, luego, si se entra nuevamente a la interrupción externa, el duty cycle volverá al 10%.
- b. Modificar los niveles de prioridad para que la interrupción por systick tenga mayor prioridad que la interrupción externa.
12. Algoritmo de antirrebote de un pulsador: Escribir un programa en C que ante la interrupción por flanco de subida del pin P0.1, configurado como entrada digital con pull-down interno, se incremente un contador de un dígito, se deshabilite esta interrupción y se permita la interrupción por systick cada 20 milisegundos. En cada interrupción del systick se testeará una vez el pin P0.1. Solo para el caso de haber testeado 3 estados altos seguidos se sacará por los pines del puerto P2.0 al P2.7 el equivalente en ascii del valor del contador, se desactivará la interrupción por systick y se habilitará nuevamente la interrupción por P0.1. Por especificación de diseño se pide que los pines del puerto 2 que no sean utilizados deben estar enmascarados por hardware. Considerar que el CPU se encuentra funcionando con el oscilador interno RC (4Mhz)
13. Calcular cuál es el tiempo máximo que se puede temporizar utilizando un timer en modo match con máximo valor de prescaler y máximo divisor de frecuencia de periférico. Especificar el valor a cargar en los correspondientes registros del timer. Suponer una frecuencia de core cclk de 50 Mhz.
14. Utilizando el Timer 1, escribir un código en C para que por cada presión de un pulsador, la frecuencia de parpadeo de un led disminuya a la mitad debido a la modificación del reloj que llega al periférico. El pulsador debe producir una interrupción por GPIO0 con flanco descendente.

15. Escribir el código que configure el timer0 para cumplir con las especificaciones dadas en la figura adjunta. (Pag 510 Figura 115 del manual de usuario del LPC 1769). Considerar una frecuencia de cclk de 100 Mhz y una división de reloj de periférico de 2.

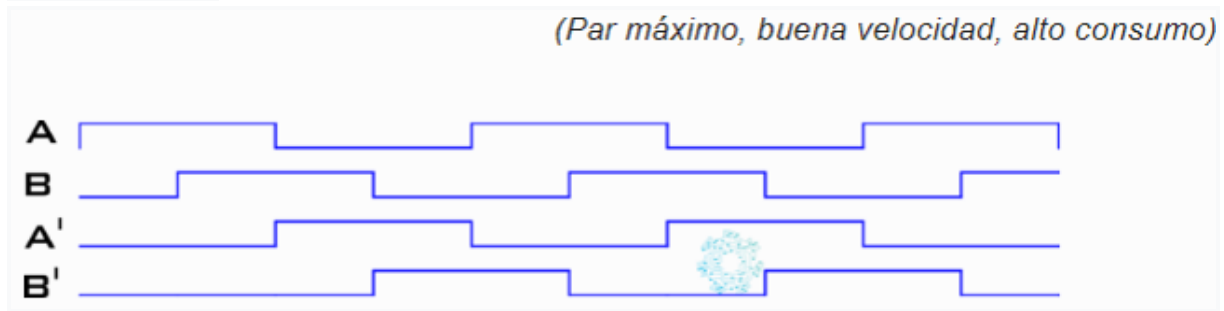


16. Escribir un programa para que por cada presión de un pulsador, la frecuencia de parpadeo de un led disminuya a la mitad debido a la modificación del pre-escaler del Timer 2. El pulsador debe producir una interrupción por EINT1 con flanco descendente.
17. Escribir un programa para que por cada presión de un pulsador, la frecuencia de parpadeo disminuya a la mitad debido a la modificación del registro del Match 0. El pulsador debe producir una interrupción por EINT2 con flanco descendente.
18. Utilizando los 4 registros match del Timer 0 y las salidas P0.0, P0.1, P0.2 y P0.3, realizar un programa en C que permita obtener las formas de ondas adjuntas, donde los pulsos en alto tienen una duración de 5 mseg. Un pulsador conectado a la entrada EINT3, permitirá elegir entre las dos secuencias mediante una rutina de servicio a la interrupción. La prioridad de la interrupción del Timer tiene que ser mayor que la del pulsador. Estas formas de ondas son muy útiles para controlar un motor paso a paso.

SECUENCIA\_A:



SECUENCIA\_B:



19. Utilizando el modo Capture, escribir un código en C para que guarde en una variable ,llamada "shooter", el tiempo (en milisegundos) que le lleva a una persona presionar dos pulsadores con un único dedo.
20. Configurar 4 canales del ADC para que funcionando en modo burst se obtenga una frecuencia de muestreo en cada uno de 50Kmuestras/seg. Suponer un cclk = 100 Mhz
21. Realizar un código en C que permita filtrar las muestras provenientes del ADC con un filtro promediador móvil de N muestras, donde N inicialmente es igual a 2 y puede ser incrementado en dos unidades cada vez que se presiona un pulsador conectado en P0.6 hasta llegar a un valor de 600. A su vez, utilizando el bit de overrun y un timer, complete el código realizado para que en caso de producirse el evento de pérdida de datos por parte del ADC, se deje de muestrear y se saque por el pin MATCH2.1 una señal cuadrada de 440 Hz. Considerar una frecuencia de cclk de 60 Mhz y configurar el ADC para obtener una frecuencia de muestreo de 5 Kmuestras/seg.